	AIR UNIVERSITY
	DEPARTMENT OF COMPUTER ENGINEERING
	EXPERIMENT NO: 3

Lab Tittle: Process Creation and Process Management

Name:	Muhammad Waqas	**Reg No:**	240756
	Muhammad Abuzar		240736

Objectives: Process Creation

LAB ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Ability to Conduct Experiment					
Ability to assimilate the results					
Effective use of lab equipment and follows the lab safety rules					

Total Marks:_____ Obtained Mark: _____

LAB REPORT ASSESSMENT:

Attributes	Excellent (5)	Good (4)	Average (3)	Satisfactory (2)	Unsatisfactory (1)
Data presentation					
Experimental results					
Conclusion					

Total Marks: _____ Obtained Marks: _____



DEPARTMENT OF ELECTRICAL & COMPUTER ENGINEERING

OPERATING SYSTEMS LABORATORY

Lab Report: 3

Tittle: Process Creation and Process Management

Name: Muhammad Waqas, Muhammad Abuzar

Reg. No: 240756, 240736

Submitted To: CE- Engr. Mam Javeria Razzaq

Date: 4 Oct, 2026

Experiment No: 3

Task 1:

Q1. Write a program which prints its PID and uses fork () system call to create a child process. After fork() system call, both parent and child processes print what kind of process they are and their PID. Also, the parent process prints its child's PID and the child process prints its parent's PID.

Code

```
GNU nano 8.7.1                    task1.c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
int main(int argc,char*argv[]){
printf("Initial Process starting with PID: %d\n",(int)getpid());
pid_t rc=fork();
if(rc<0){
fprintf(stderr,"fork failed\n");
exit(1);
}else if(rc==0){
printf("I am the CHILD process.\n");
printf("My PID is: %d\n",(int)getpid());
printf("My Parent's PID is: %d\n",(int)getppid());
}else{
printf("I am the PARENT process.\n");
printf("My PID is: %d\n",(int)getpid());
printf("My Child's PID is: %d\n",(int)rc);
wait(NULL);
}
```

Terminal(Output)

```
vboxuser@Ubantu: ~/Music/lab3
vboxuser@Ubantu:~$ cd Music
vboxuser@Ubantu:~/Music$ mkdir lab3
vboxuser@Ubantu:~/Music$ cd lab3
vboxuser@Ubantu:~/Music/lab3$ nano task1.c
vboxuser@Ubantu:~/Music/lab3$ gcc task1.c -o task1
vboxuser@Ubantu:~/Music/lab3$ ./task1
Initial Process starting with PID: 4642
I am the PARENT process.
My PID is: 4642
My Child's PID is: 4643
I am the CHILD process.
My PID is: 4643
My Parent's PID is: 4642
vboxuser@Ubantu:~/Music/lab3$ nano task1.c
```

Task 2

Q2. Write a program which uses fork () system-call to create a child process. The child process printsthe contents of the current directory and the parent process waits for the child process to terminate

Code

vboxuser@Ubantu: ~/Music/lab3 — nano task2.c
~/Music/lab3

GNU nano 8.7.1 task2.c

```
#include<stdio.h>
#include<unistd.h>
#include<stdlib.h>
#include<sys/wait.h>
#include<sys/types.h>
int main(){
pid_t pid;
int status;
pid=fork();
if(pid<0){
fprintf(stderr,"Error has Occured Fork Failed\n");
exit(1);
}else if(pid==0){
printf("This is the Child Process and its PID is %d\n",(int)getpid());
execlp("ls","ls","-l",NULL);
printf("An Error has Occured in Execlp\n");
exit(1);
}else{
int wc;
wc=wait(NULL);
printf("This is the Parent Process of %d Waiting for the child process.\nWC:%d\
}
return 0;
}
```

Terminal(Output)

```
vboxuser@Ubantu:~/Music/lab3$ nano task2.c
vboxuser@Ubantu:~/Music/lab3$ gcc task2.c -o task2
vboxuser@Ubantu:~/Music/lab3$ ./task2
This is the Child Process and its PID is 5182
total 40
-rwxrwxr-x 1 vboxuser vboxuser 16288 Oct  4 14:52 task1
-rw-rw-r-- 1 vboxuser vboxuser   548 Oct  4 14:54 task1.c
-rwxrwxr-x 1 vboxuser vboxuser 16288 Oct  4 15:02 task2
-rw-rw-r-- 1 vboxuser vboxuser   546 Oct  4 15:02 task2.c
This is the Parent Process of 5182 Waiting for the child process.
WC:5182
PID:5181
vboxuser@Ubantu:~/Music/lab3$
```

Task 3

Q3. Write a program where parent process takes command line arguments a list of letters, the child process uses this list to form 5 random words and checks whether they exist in the dic.txt file. If the word exists, the child prints successfully found the word else prints word not found in file. Meanwhilethe parent process waits for the child process to terminate and after the child process terminates it prints a message. The above programs should use Linux calls like fork, exit and wait.

Code

```
  GNU nano 8.7.1
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>
#define WORD_COUNT 5
#define WORD_LENGTH 5
#define MAX_LINE_LENGTH 100
void generate_random_word(char *letters[],int letter_count,char *word)
{
for(int i=0;i<WORD_LENGTH;i++)
{
int index = rand() % letter_count;
word[i] = letters[index][0];
}
word[WORD_LENGTH]='\0';
}
int word_exists_in_file(char *word)
{
FILE *file=fopen("dict.txt","r");
if(!file)
{
printf("Failed to open dict.txt");
exit(1);
}
```

```
char line[MAX_LINE_LENGTH];
while(fgets(line,sizeof(line),file))
{
line[strcspn(line,"\n")]='\0';
if(strcmp(line,word)==0)
{
fclose(file);
return 1;
}
}
fclose(file);
return 0;
}
int main(int argc, char *argv[])
{
if(argc<2)
{
printf("Usage: %s <list of letters>\n",argv[0]);
exit(1);
}
pid_t pid=fork();
if(pid<0)
{
printf("Fork Failed\n");
exit(1);
}
printf("Fork Failed\n");
exit(1);}
if(pid==0){
srand(time(NULL));
printf("Child process started. The PID of child is %d\n",(int)getpid());
for(int i=0;i<WORD_COUNT;i++){
char word[WORD_LENGTH + 1];
generate_random_word(&argv[1],argc-1,word);
if(word_exists_in_file(word)){
printf("Successfully found the word: %s\n", word);}
else{
printf("Word not found in file: %s\n", word);}}
printf("Child process finished.\n");
exit(EXIT_SUCCESS);}
else{
int wc=wait(NULL);
printf("Parent process:wc=%d PID=%d.Child has terminated.\n",wc,(int)getpid());}
return 0;}
```

Terminal(Output)

```
vboxuser@Ubantu:~$ gcc Task3.c -o j
vboxuser@Ubantu:~$ ./j a b c d e
Child process started. The PID of child is 6394
Word not found in file: ccded
Word not found in file: bbeea
Word not found in file: dccad
Word not found in file: aeeaa
Word not found in file: ddaed
Child process finished.
Parent process:wc=6394 PID=6393.Child has terminated.
vboxuser@Ubantu:~$
```

Conclusion:

In this experiment, we learned how Linux manages processes using fundamental system calls. By writing C programs, we observed how the fork() function creates a brand new child process with its own separate ID. We used getpid() and getppid() to track down the exact parent and child process IDs during execution. Using the 'wait()' system call allowed us to make sure the parent process pauses until the child finishes its work. We also implemented execvp() to run external system commands like ls directly from a child process. In the final task, we combined these concepts to generate random words in a child process while reading from an external file. We also practiced managing running programs using shell commands like 'ps', 'jobs', fg, and bg. Overall, this lab gave us hands-on experience in process creation, synchronization, and basic process management in Linux.